

The EEZZ Project

Albert Zedlitz

November 2023

Abstract

This document describes part of the infrastructure development for an EEZZ archive, mainly the user interface part. EEZZ implements a system designed for packaging and displaying digital content. As an example, consider distributing a movie. Associated information such as ratings, descriptions, and previews, along with the encrypted movie file and purchase information for the decryption key, are bundled within the EEZZ archive. The EEZZ environment allows the archive to be shared without restrictions in a public cloud, managing individual keys for each subscribed user using an 1:N decryption scheme. This would allow digital content to be published and purchased independent from any platform, opening a complete new purchase model for artists in music or movie industry, allowing a 1:1 transaction with the customers.

Recognizing the browser as a natural platform for displaying multimedia content, I began investigating how to extend its functionality to directly show an EEZZ archive, while interacting with the encryption key store, intended to reside on a mobile device. I start searching for existing infrastructure projects, but none of them could fulfill all my requirements. So I started my own project.

I put my focus on decoupling user interface aspects and data management, because I wanted to involve a professional user interface designer any time in the project, without hand over or synchronize the back-end implementation.

I wanted to minimize usage of JavaScript and server-side scripting, which are always difficult in maintenance. In such environments I met sometimes up to four languages in different layers of the execution stack, all with their own debugging runtime systems and code resources. Implement changes or fixing an issue in such an environment could be a real nightmare.

With these principles in mind, let's have a closer look, how the browser could turn into a multi purpose application interface. As a result I could show a first glance of how an compound document like the EEZZ archive could be managed. Compared to traditional approaches I could find significant advantages in many aspects in software development.

Even though I started with this very special use case, I think the result is in no way restricted to this and may be pretty much useful also for other applications. So let's dive into it.

1 Introduction

While graphical user interfaces are absolute necessary nowadays for any project, it also addresses several key issues that could face you, particularly if you are without specialized training in user interface (UI) design. Especially me as a back-end and systems-level close-to-the-hardware developer, I often struggle when tasked with creating intuitive, visually appealing UIs. My results in this case were almost always a poor user experiences, which negatively affected even my most powerful command-line applications.

In the process of establishing an user interface front end, I investigated and worked with many third-party libraries like *Qt* or *Microsoft Visual C++ (MSVC)*. However, incorporating these libraries into a project introduced several critical issues:

- **Platform Coverage:** Third-party libraries often have varying levels of support for different operating systems. *Qt*, for example, is known for its cross-platform capabilities, but it can add significant overhead to projects. On the other hand, *MSVC* is closely tied to the Windows platform, making it difficult to maintain compatibility across Linux, macOS, and other environments. Choosing a library can lead to trade-offs between compatibility, performance, and complexity.
- **License Models:** Many third-party libraries come with complex licensing models that developers have to consider. For instance, *Qt* offers both open-source and commercial licensing, which can impose constraints on how the software can be used, distributed, or modified. *MSVC*, as a proprietary tool, comes with its own set of licensing terms that can limit flexibility and impact project costs. Misunderstanding or overlooking these licensing requirements can lead to legal issues or unexpected expenses down the road.
- **Project Binding:** In most cases your code is heavily intermixed by library calls to manage interface input and data transfer. This makes your code harder to maintain, extend or test. Many issues like strings, which are not translated or graphic elements which will not scaled properly are detected only very late in the project and are very hard to correct. There may be many layers before output, each of which need some expertise and maintenance. You can only hope, that the support will last as long as your project needs it.

As you could see, the integration of user interfaces presented more challenges to me than opportunities. That was the starting point, where I had the idea for a complete new approach. I wanted to attach my command-line applications to *HTML* outputs, using *WebSocket* technology.

In practice this new approach speed up time to customer, because I could separate the development to a second teams, without any interference. Driving tests became much easier and resulted into better quality and performance. The examples given in this article show, that you could achieve some nice results with minimal efforts.

2 The User Interface

From TTable to HTML

Before getting into details, let's have a preview on the usage to motivate the further work. The EEZZ project, which implements this environment, consists of the following parts.

- The application interface to collect data (*eezz.table.TTable*)
- The web-socket handler (*eezz.websocket.TWebSocketClient*)
- The HTML rendering service (*eezz.http_agent.THttpAgent*)
- The JavaScript handler (*websocket.js*)
- The HTTP server (*eezz.server.TWebServer*)

The *HTTP* server has no special tasks to do and could be replaced by Apache or NGInx.

As an example how these elements are fit together, let's have a look at the source in Figure 1, which shows the basic setup after installation of the EEZZ extension and a small example project, which displays the content of a local directory written in *Python*. The output in the console window shows the project before any graphical user interface is active. This is the part for collecting data.

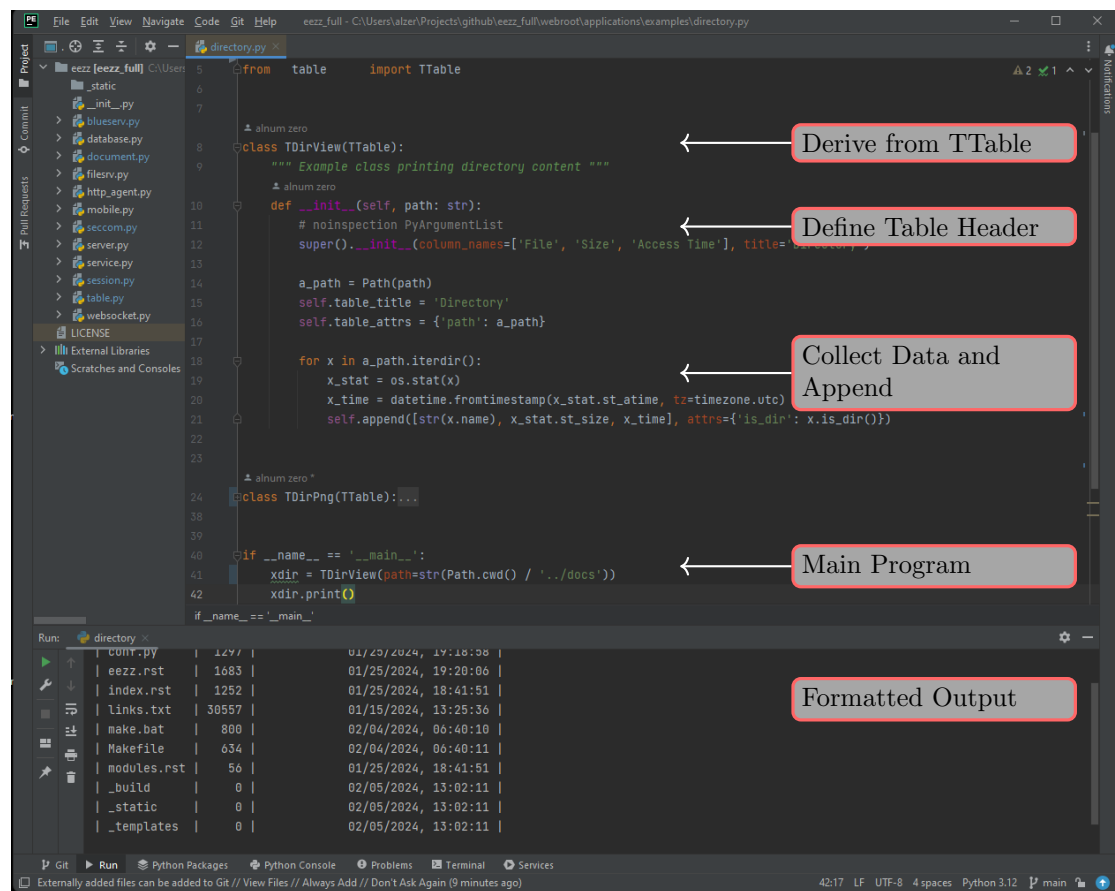


Figure 1: Example for Directory Listing

First thing you should notice is, that there is no hint whatsoever for any user interface. Except perhaps, that the project has to reside in the web-root directory of an *HTTP* server. A central point for the further development is the class *TTable*, which is already capable to print a nice ASCII-table to console, using the python data types for formatting. You could easily add your own data types with special format rules.

Regarding the code implementation project for a simple output-only scenario, this task is done and you could seal your work and go home. No further changes necessary for this part.

The central feature of *TTable* its ability to organize formatted output which is the basis for generating a valid *HTML* stream.

Prepare HTML Page for Output

The HTML code in listing 1 shows a setup to call your application and display the data in a browser window. You see a rudimentary HTML page with the special attribute *data-eezz* on some nodes. These eight lines are really all you need to this first try.

```
1 <!DOCTYPE html>
2 <head><title>Title</title>
3   <script data-eezz="template:websocket"></script></head>
4 <html><body>
5   <h1>Simple Table</h1>
6   <table data-eezz="assign:examples.directory.TDirView({query.path})"></table>
7 </body>
8 </html>
```

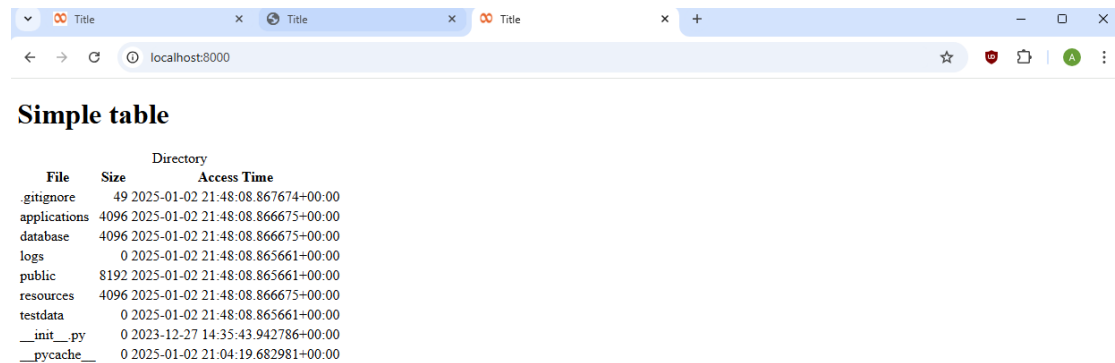
Listing 1: index.html for Directory Listing

The content of attribute *data-eezz* is compiled into statements, as described in the definition of the EEZZ syntax in Listing 2. In the header section, there is the “*template*” statement in the “*<script>*” node tag, which creates an include statement for the EEZZ web-socket implementation. For the “*<table>*” node tag, the “*data-eezz:assign*” statement connects the example *TDirView(TTable)* to the *HTML* and the rest is done by the framework.

With the curly brackets you have access to the following:

- URL query section (only for the assign): *{query.*}*
- Attributes of *TTable*: *{table.*}*
- Attributes of *TTableRow* (only for *tr* nodes): *{row.*}*
- Attributes of *TTableCell* (only for *td* or *th* nodes): *{cell.*}*

For the next steps you would need to deploy this file, configure and restart the HTTP server, in this case to listen on port 8000, and the result is available in your favorite browser.



The screenshot shows a web browser window with the address bar set to `localhost:8000`. The page title is "Simple table". The content is a directory listing table with the following data:

Directory		
File	Size	Access Time
.gitignore	49	2025-01-02 21:48:08.867674+00:00
applications	4096	2025-01-02 21:48:08.866675+00:00
database	4096	2025-01-02 21:48:08.866675+00:00
logs	0	2025-01-02 21:48:08.865661+00:00
public	8192	2025-01-02 21:48:08.865661+00:00
resources	4096	2025-01-02 21:48:08.866675+00:00
testdata	0	2025-01-02 21:48:08.865661+00:00
__init__.py	0	2023-12-27 14:35:43.942786+00:00
__pycache__	0	2025-01-02 21:04:19.682981+00:00

Figure 2: Browser Output: `http://localhost:8000`

Nothing too spectacular, but this demonstrate the basic idea. In the following I will show the entire setup and all the possibilities to interact with the output for user defined formatting and scripting.

In each project software engineers and user interface experts should generate such a small *HTML* page, to define the interface as a contract. This defines the classes with all methods and arguments to be used. Now both could start working in their own team.

2.1 Conclusion and Perspective

The takeaway from this chapter is, that it is quite easy and convenient in the EEZZ environment to add a user interface to an existing command line driven application. Using a browser has also an advantage, that you could decide any time to switch to a network based solution. As developer, you could concentrate on programming in Python. No *SQL*, no GUI-packages, no *JavaScript* nor any of its wrappers or frameworks to interoperate with. We have a clear cut between UI related staff and the rest.

In the following sections I will show, that I'm not restricted on tables. There is a variety of possible layouts. For example tree views, grid-container and input-forms. It will extend this introduction of the basis concept, to get into depth. So I will only mention, that it's also possible to handle more complex elements like SVG, including libraries like *Chart.js*, introduce motion and more complex user interaction.

3 Behind the Scenes

3.1 EEZZ Grammar

The resources to build up page and to establish the connection to the *Python* code are located in a directory parallel to the custom pages. In here you find the *JavaScript* package *websocket.js*, a template for a table layout *template.html* and a grammar shown in in Listing 2 for the EEZZ interface.

The EEZZ interface uses only the user property namespace *data-eezz* on some *HTML* node elements. This has the advantage, that the *HTML* page could be designed independent from the application and that there is no intermixing with script and layout. There is also the possibility to work with design time data, so there should be no surprises, when matching the parts, application and user interface, together and there should be no need for sophisticated debugger to get things running.

?start	:	list_statements	
?list_statements	:	[statement (";" statement)*]	-> list_statements
?statement	:	"onload"	-> onload_section
		"oninit"	-> oninit_section
		"assign"	-> table_assignment
		"event"	-> funct_assignment
		"update"	-> update_section
		"task"	-> task_section
		"eventhandler"	-> add_event_handler
		"environment"	-> list_assignment
		"template"	-> template_section
		"style"	-> set_style
		"ontest"	-> set_style
list_assignment	:	(STRING "=" STRING) ("," STRING "=" STRING)*	

```

value_qualified      : STRING | array_selector ( "." QUALIFIED_STRING )?
array_selector       : STRING "[" STRING "]" ( "." QUALIFIED_STRING )?

function_call        : QUALIFIED_STRING "(" list_arguments ")"
list_arguments       : [ named_argument ( "," named_argument )* ]
named_argument       : QUALIFIED_STRING "=" (STRING_VALUE_EQ | escaped_string | SIGNED_NUMBER | ARRAY_DESCR) -> assignment

list_updates         : [ update_item ( "," update_item )* ]
?update_item         : QUALIFIED_STRING ( "=" function_call )? -> update_function
                     | QUALIFIED_STRING "=" STRING_VALUE_EQ -> assignment
                     | function_call -> update_task

%import common.ESCAPED_STRING
%import common.CNAME
%import common.SIGNED_NUMBER
%import common.WS
%ignore WS

STRING               : CNAME
QUALIFIED_STRING     : STRING ( "." STRING )*
SELECT_ARRAY         : STRING "[" STRING "]" ( "." QUALIFIED_STRING )?
FORMAT_STRING        : "{" (QUALIFIED_STRING | SELECT_ARRAY) "}"
STRING_VALUE_EQ      : QUALIFIED_STRING | FORMAT_STRING | SELECT_ARRAY
UNIT_NUMBER          : SIGNED_NUMBER CNAME
ARRAY_DESCR          : "[" (WS)* QUALIFIED_STRING [(WS)* "," (WS)* QUALIFIED_STRING (WS)*]* "]"

escaped_string       : ESCAPED_STRING -> escaped_str

```

Listing 2: EEZZ Extension Grammar: eezz.lark

As you can see, the grammar description is quit small. At the end it is compiled to a json format, which is attached to the elements attribute *data-eezz-json*.

3.2 Templates

A template is a node, which is used for rendering table rows to *HTML* elements. So where are the templates in the first example shown in Listing 1?

The answer is, that I wanted to optimize the interaction with the back-end, which required all the table child elements to be explicit available during rendering. So I inject missing parts using the definition shown in Listing 3, which contains all the required templates.


```

10 <table id="{table.id}" data-eezz="assign: {table.module}, name: {table.name}, columns: [{table.columns}]">
11 <caption data-eezz="template: table">{table.title}</caption>
12 <thead>
13 <tr data-eezz="template: row; environment: match = header">
14 <th class="clzz_th"
15 data-eezz="
16     template: cell;
17     event: do_sort(column={cell.index});
18     update: this.tbody">{cell.value}</th></tr>
19 </thead>
20 <tbody>
21 <tr data-eezz="
22     template: row;
23     environment: match = body;
24     event: on_select(row={row.row_id})">
25 <td class='clzz_{cell.type}' data-eezz="template: cell">{cell.value}</td></tr>
26 </tbody>
27 <tfoot>
28 <tr data-eezz="
29     template: reference(table);
30     style: visibility={table.visible_navigation}">
31 <td style="text-align:center" colspan="4">
32 
35 <map class="clzz_navigation" name="table-navigation">
36 <area shape="rect" coords=" 0, 0, 40, 20"
37 data-eezz="event: navigate(where_togo = 3); update: this.tbody"/>
38 <area shape="rect" coords=" 40, 0, 80, 20"
39 data-eezz="event: navigate(where_togo = 2); update: this.tbody"/>

```

Listing 3: Extract of template.html

There are row and cell templates. Row templates are used to design different layouts based on the value of *TTable.row_type* attribute, which is referenced in the page as *data-eezz:match*. Default values are *header* and *body* for *thead* and *tbody* table child elements respectively. The *tfoot* rows are not included in this template concept and are considered to be constant.

3.3 Events

The following events are implemented in the injected definition, but you could also define own event methods:

- *TTable.do_sort* for each column in the thead section
- *TTable.do_select* for each row in the tbody section

The first statement potentially alters the table content and needs an update directive, to make changes visible. The select statement could be handled without any changes in the data, so an update statement is optional.

All events are executed in a separate thread to prevent blocking the user interface, if there is a long running method. You could issue the statement *"process:sync"* to force execution synchronously.

3.4 Time Laps View

The time laps diagram below Figure 3 shows, that even the program flow is very simple. I put the *TDirView* from the example above in the rightmost row. Here you could imagine your python *TTable* class.

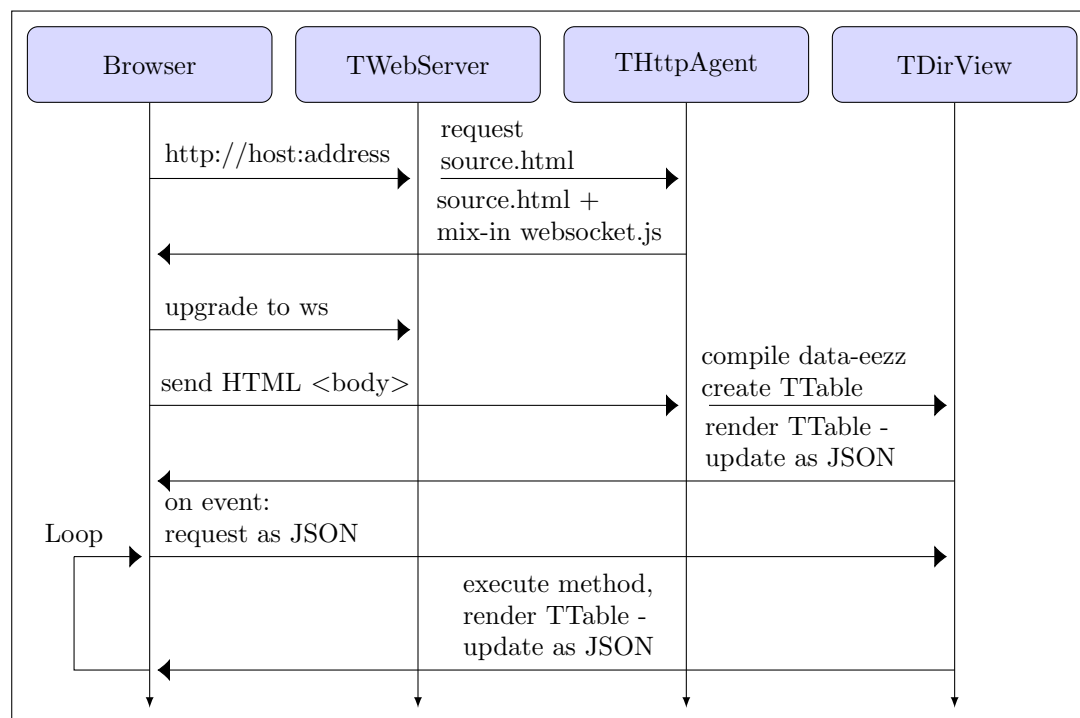


Figure 3: Time lapsed diagram

3.5 Conclusion

This chapter showed the basic concept of the EEZZ extension, the data flow and the default settings. It gave a short overview over the in- and output features and interaction between server and client.

The EEZZ extension allows you to connect *HTML* element attributes with the programming logic of a back-end system. It does not introduce another wrapper to *JavaScript* nor does it introduce a new programming language.

4 Features

There are some features, which I implemented for better efficiency in project development. The first addresses the handling of mass elements of the same kind. The second shows an alternative for polling data. And last not least I did some work to make life with database access easier.

4.1 Arrays as Value Type

It's possible to define an array for a cell in the *TTable* row definition. In this case the tree view would take the first element of such an array for display and is therefor for less use. In a grid view however, it allows you to create a set of elements of the same kind, creating a grid element for each entry.

Take for example the calculator app in Listing 4. The *TCalc(TTable)* class defines two columns (*numpad* and *op*) and two new methods. The method *key_pad_input* organizes incoming digits to build up numbers and *key_op_input* executes math on two of these numbers, returning the result. All possible digits are defined in an array for column *numpad* and the operators in an array for column *op*

```

4 class TCalc(TTable):
5     def __init__(self):
6         self.number_input: int = 0
7         self.stack: list = list()
8         self.op: str = ''
9
10        super().__init__(column_names=['numpad', 'op'], visible_items=1)
11        self.append([[1,2,3,4,5,6,7,8,9,0], ['+', '-', '*', '=']])
12
13        def key_pad_input(self, key) -> bytes:
14            self.number_input += 10
15            self.number_input += int(key)
16            return f'{self.number_input}'.encode('utf8')
17
18        def key_op_input(self, key) -> bytes:
19            result: float = 1.0 * self.number_input
20
21            self.stack.append(self.number_input)

```

Listing 4: Calculator

The HTML page references such cells with the detail property *template:cell(<name of the column>.detail)*. The advantage of this approach is, that you need only two templates for 14 buttons as shown in Listing 5.

```

27 <div style="display: grid; grid-template-columns: repeat(3, 30px); gap:5px;">
28 <input type = 'button'
29 value ="{{detail.value}}"
30 data-eezz ="
31     template: cell(numpad.detail),
32     event :   get_header_row(),
33     update:   display.innerHTML = key_pad_input(key={{detail.value}})"/>
34 </div>
35 <div style="display: grid; grid-template-rows: repeat(5, 30px); gap:5px;">
36 <input type = 'button'
37 value ="{{detail.value}}"
38 data-eezz ="
39     template: cell(op.detail),
40     event:   get_header_row(),
41     update:   display.innerHTML = key_op_input(key={{detail.value}})"/>

```

Listing 5: Calculator Layout

As a result we have created a calculator as a working application as shown in Figure 4

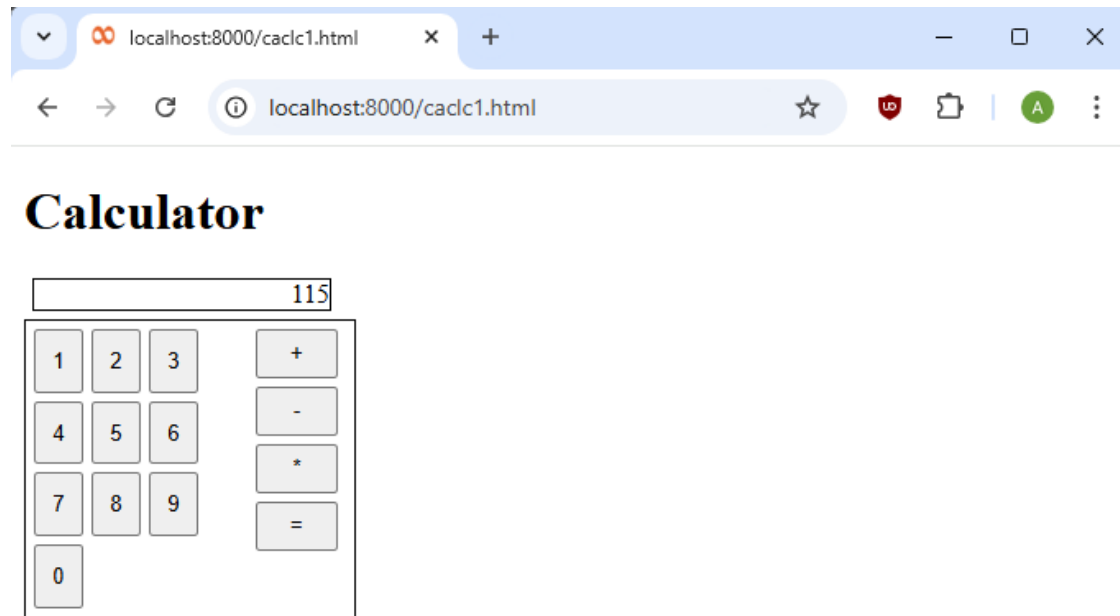


Figure 4: Calculator with Cell-Array

It would be the same setup for a boards used by Minesweeper, Chess or Sudoku. Another usage could be the transmission of coordinates for dynamic generated charts. This is also my preferred way to implement a dynamic selector element. The selected cell value is just one of the elements in the list in the details.

```

12 <div style = "width:200px; border: 1px solid black;"
13 class = "clzz_grid"
14 id = "foods_fruits"
15 data-eezz = 'assign: examples.selector.TSelect()'>
16
17 <div data-eezz = "
18     template: row;
19     environment: match = body">
20 <select id = "mySelect"
21     data-eezz = "
22         template: cell(select);
23         event : do_select_fruit(fruit = this.value)">
24 <option
25     data-eezz = "template: cell(select.detail)"
26     value="{detail.value}">{detail.value}</option>

```

Listing 6: Dynamic Select

Here are the some lines in the Python script to provide the data and process the input event:

```

1   TSelect(TTable)
2   def __init__():
3       TTable.column_names = ['select']
4       TTable.append(['Apple', 'Banana', 'Oranges'])
5       ...
6
7   def do_select_fruit(fruit):
8       ...

```

Listing 7: Python Script for Dynamic Select

With this concept it's also possible to provide data for a *Chart.js* application. The column names are JSON keys in the argument of the provided function. This approach is quite generic and allows output of data in various scenarios.

```

14  class TCharts(TTable):
15      def __init__(self):
16          super().__init__(column_names=['x', 'y', 'colors', 'title'])
17
18      def get_data(self) -> TTableRow:
19          x_row = self.append([["Italy", "France", "Spain", "USA", "Argentina"],
20                               [55, 49, 44, 24, 15],
21                               ["red", "green", "blue", "orange", "brown"],
22                               "World Wine Production 2018"])
23          return x_row

```

Listing 8: Python Script for Chart

```

6   <script>
7       function draw_chart(a_json_args) {
8           new Chart("myChart", {
9               type: "bar",
10              data: { labels: a_json_args.x,
11                        datasets: [{
12                            backgroundColor: a_json_args.colors,
13                            data: a_json_args.y}]
14              },
15              options: {
16                  legend: { display: false },
17                  title: { display: true,
18                          text: a_json_args.title }
19              }
20          });
21      }
22  </script>

```

Listing 9: Chart Script

```

25  <body>
26  <canvas id="myChart" style="width:100%;max-width:600px">Canvas not supported</canvas>
27  <button onclick='draw()'>click</button>
28
29  <div class="clzz_grid" id="Chart"
30      data-eezz='
31      assign: examples.board.TCharts();
32      oninit: get_data();
33      update: draw_chart(title={row[title].value}, x={row[x].detail_values}, y={row[y].detail_values}, colors={row[colors].
34      <-> detail_values} )'>

```

Listing 10: Chart HTML

The assign statement in Listing 10 connects the Python script and the directive *oninit:get_data* provides the data. The *update* statement triggers the *JavaScript* function after the page is loaded. The argument of the script is a *JSON* structure, with the the keys corresponding the the table

column names as shown in Listing 9. As a result there is a solution to transfer data to an HTML graph as shown in Figure 5

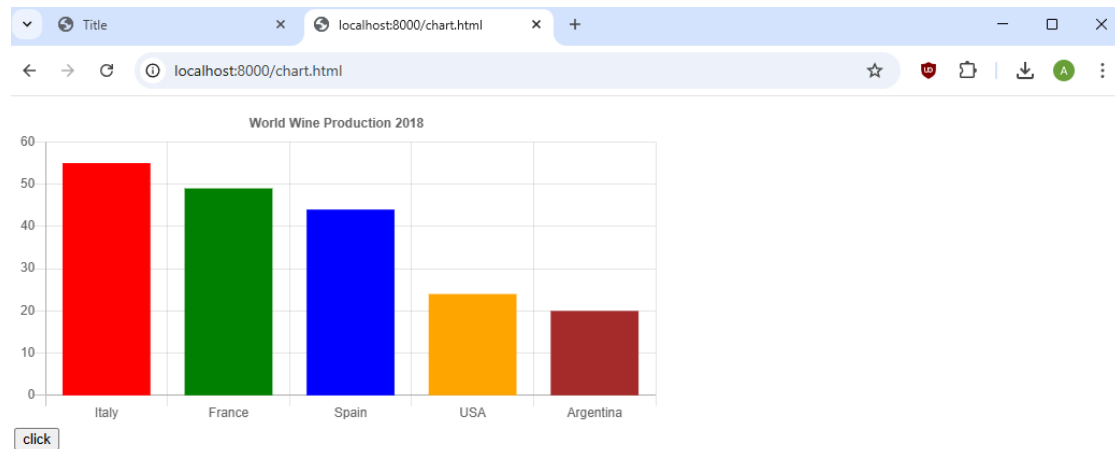


Figure 5: Chart

4.2 Enumeration as CSS-Type

Using a tuple for any of the column names allows you to set the name with the first parameter and the type with the second. The default type is set to `"_auto"`, in which case the type is evaluated with the first row inserted. All these auto generated type names start with underscore like `"_int"` or `"_str"`. The type could be any string and is primarily used to define format and color.

```
1 TTable.column_names = [('Label', '_enum'), 'Message']
```

Listing 11: Declaration of Type Enum

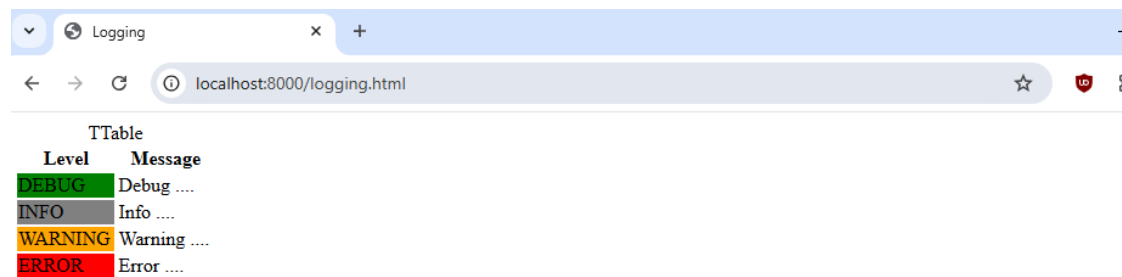
Using the type `"_enum"` the type returned is the value of the cell, which allows formatting based on the value displayed. To do so, we define the class of the *HTML* element as `class = clzz_{cell.type}` and set for example the color in the *CSS* part.

```

1  <style>
2  .clzz_DEBUG {
3      background-color: green;
4  }
5  .clzz_INFO {
6      background-color: gray;
7  }
8  .clzz_WARNING {
9      background-color: orange;
10 }
11 .clzz_ERROR {
12     background-color: red;
13 }
14 </style>

```

Listing 12: CSS Type to Color



The screenshot shows a web browser window with the address bar displaying 'localhost:8000/logging.html'. Below the browser window, a table is displayed with the following structure:

TTable	
Level	Message
DEBUG	Debug
INFO	Info
WARNING	Warning
ERROR	Error

Figure 6: Colors by Enumeration

A more sophisticated way to evaluate the type is using a callable as second parameter. In this example a pressure value is displayed in different colors. It's possible to define other functions for selection or color-gradients instead of fixed values.

```

1  TTable.column_names = [('pressure', lambda x: 'WARNING' if x < 80 else 'ERROR' if x > 120 else 'INFO')]

```

Listing 13: Colors by Function

With such a function programming a chess board is straight forward. As shown in Listing 14 the color of the chess board is calculated as follows:

$$color(x) = \begin{cases} (x + x \% 8) \% 2 == 0 & \Rightarrow white \\ else & \Rightarrow black \end{cases}$$

```

4 class TChess(TTable):
5     def black_or_white(self, a_num):
6         x1, x2 = divmod(a_num, 8)
7         return 'black' if (x1 + x2) % 2 else 'white'
8
9     def __init__(self):
10         super().__init__(column_names=[('Field', self.black_or_white)])
11         self.append([[i for i in range(64)], row_type='body'])

```

Listing 14: Python Script for Chess Board

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47
48	49	50	51	52	53	54	55
56	57	58	59	60	61	62	63

Figure 7: Chess Board

4.3 Push Service

It is possible to specify a task function as show in the following. This starts the given function in a thread loop, which enable the program to push any data any time to the browser.

```

38 <div class='clzz_grid'
39     id="WebCamClass"
40     data-eezz='
41         assign: examples.webcam.TCamera();
42         task:  webcaming.src = read_frame()'>
43 </div>
44
45 <img id='webcaming' />

```

Listing 15: WebCam as Push Service

In the example in Listing 15 the *img* element on the page is changed whenever the function *read_frame* returns a picture. The implementation of this function is straight forward as shown in Listing 16.

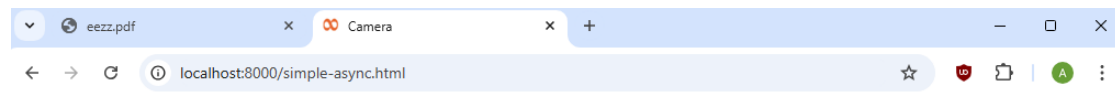

```

8 class TCamera(TTable):
9     def __init__(self):
10         self.cam = cv2.VideoCapture(0)
11         super().__init__(column_names=['camera'])
12
13     def read_frame(self) -> bytes:
14         time.sleep(5)
15         ret, frame = self.cam.read()
16         ret, buffer = cv2.imencode('.png', frame)
17         return BytesIO(buffer).getvalue()

```

Listing 16: WebCam Implementation

This is the entire project and the result is shown in Figure 8. I used *OpenCV* to generate a *PNG* stream buffer, waiting for some seconds between each frame and would now be able to monitor my room from remote, if I grant access to the URL.



Push Service



Figure 8: WebCam as Push Service

4.4 Database Access

The derived class *TDatabaseTable(TTable)* manages database access. It encapsulates database access transparently, with the same properties and interfaces as the base class. Access to the database with this class will add the options *limit* and *offset* to any *SQL* command, to reduce the data flow for the entire application stack to the absolute minimum and still holding track on the entire result set and cache data. Most common mistakes in database handling could be avoided this way.

4.4.1 Defining the Fields

The database columns are defined as tuples. The first element is the name, the second is the database type and the third is the optional python type as described in the chapter above. A star at the beginning of the type indicates a primary key.

The following statement generates a database table *Directory* in the database *test.db*. Both are created, if not already exists. The primary key is the column *'File'*

```
1 database_table = TDatabaseTable(  
2     database_name = 'test.db',  
3     title = 'Directory',  
4     column_names = [('File', '*text'), ('Size', 'integer'), ('AccessTime', 'datetime')])
```

Listing 17: Database Column Description

4.4.2 Access Database

New inserted data will be transferred to the database only after the call to *TDatabaseTable.commit*. Retrieving the data is done with a call to *TDatabase.get_visible_items*.

The method *TTable.filter_rows* encapsulates the select statement on the database. The parameter is a list of lists. The inner lists generates combined *'AND'* statements, which are then collected to *'OR'* statements.

```
1 # The following statement:  
2 database_table.filter_rows([(['Size', TOperator.GT, '0']), ('Size', TOperator.LT, '20000')], [['File', TOperator.LIKE, '%py']])  
3  
4 # Would result into:  
5 select * from Directory where (Size > 0 and Size < 20000) or (File like %py)
```

Listing 18: Database Select

This encapsulation allows to define a result set for frequent access in the table navigation, keeping an consistent internal cache. Calling *filter_rows* resets the internal buffers.

4.5 Logging

With one line you have access to the logger output of one request. You have to specify the update to the elements you want to monitor with the attribute: *data-eezz=update:eezz_log_table*. This is helpful, because you could focus on just one window, monitoring the performance and the execution.

```
1 <table id = "eezz_log_table" data-eezz="assign: eezz.websocket.TLogger()"></table>
```

Listing 19: Activate the Logger

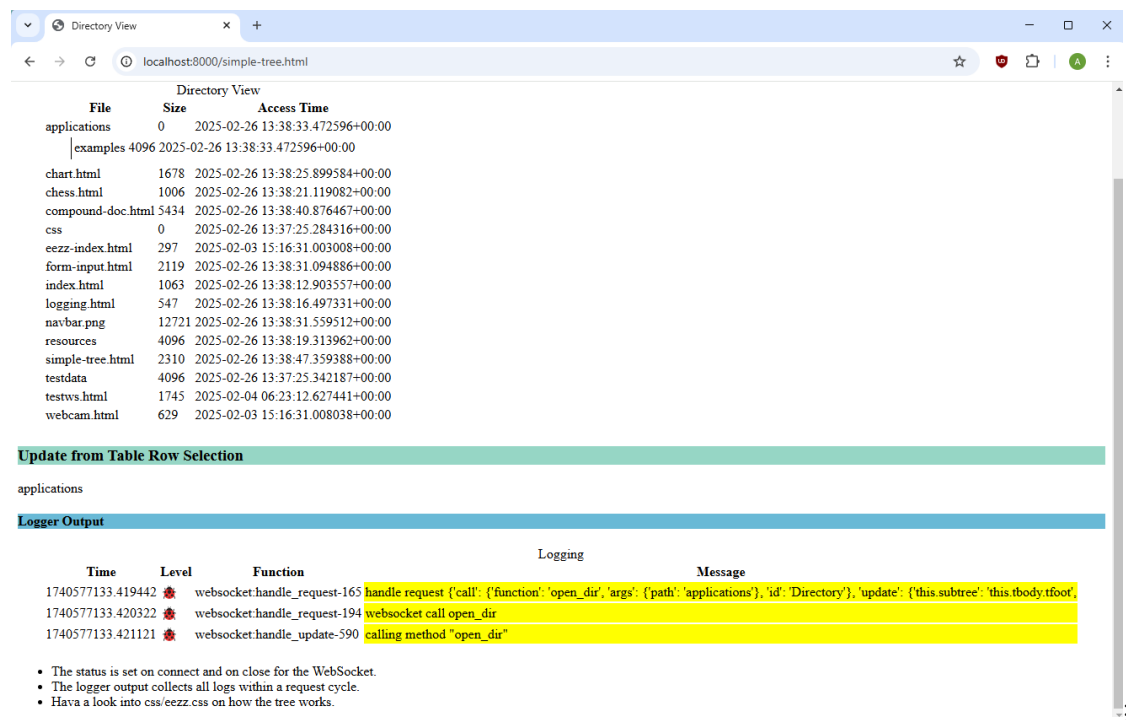


Figure 9: Logging Output

5 Improve the Output

5.1 Using Meta Data

The Listing 20 shows how to access new incoming elements for rework. Just implement the abstract method `eezz.on_update`, which is called by the framework.

Let's start to improve the time format. You need no access to the generating *Python* program to achieve this. The *Python* `datetime` object is rendered to the `timestamp` attribute of the element. We could access the value directly and convert it to a local time.

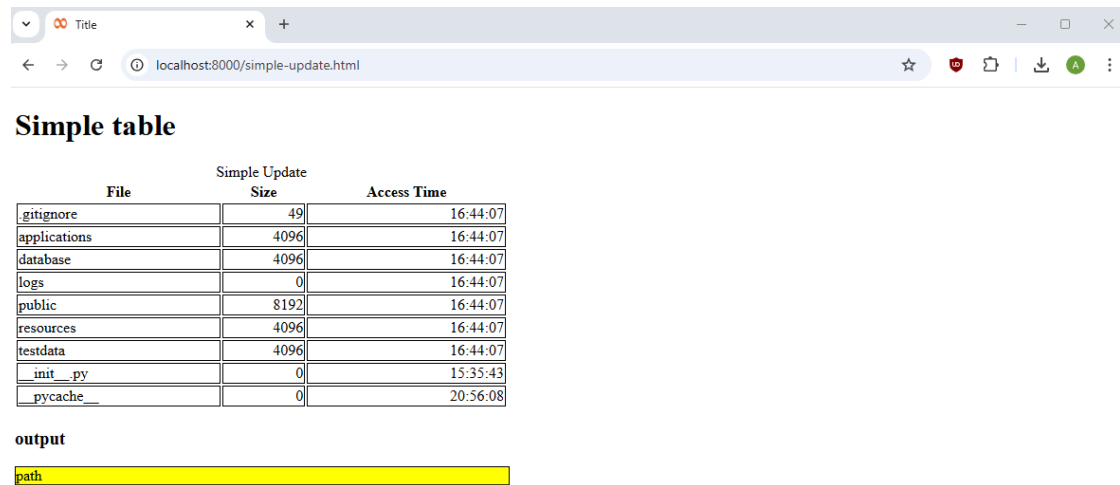
```

20 <script type="text/javascript">
21   eezz.on_update = (a_element) => {
22     x_element = document.getElementById(a_element);
23     x_list = x_element.querySelectorAll("td[timestamp]");
24
25     for (var idx in x_list) {
26       var x_time = x_list[idx];
27       var x_timestamp = x_time.getAttribute('timestamp');
28       var date = new Date(x_timestamp * 1000);
29       x_time.innerHTML = date.toLocaleTimeString();
30     }
31   }
32
33 </script>

```

Listing 20: Head of simple-update.html

As a result the time output could be changed, after update. This gives the designer the full access and utmost flexibility on any data, which is fetched from an application.



The screenshot shows a web browser window with the address bar displaying 'localhost:8000/simple-update.html'. The page title is 'Simple table'. Below the title is a table with the following data:

File	Size	Access Time
gitignore	49	16:44:07
applications	4096	16:44:07
database	4096	16:44:07
logs	0	16:44:07
public	8192	16:44:07
resources	4096	16:44:07
testdata	4096	16:44:07
__init__.py	0	15:35:43
__pycache__	0	20:56:08

Below the table, there is a section labeled 'output' with a yellow box containing the text 'path'.

Figure 10: Browser Output: <http://localhost:8000/simple-update.html>

5.2 Matching Templates

The fragment in listing 21 shows, how to use the *data-eezz:match* attribute to specify the layout of a row, according to a given data type. If *data-eezz:match* is set to "is_dir" a directory icon is inserted at the start of the row and a file icon if the value is set to "is_file".

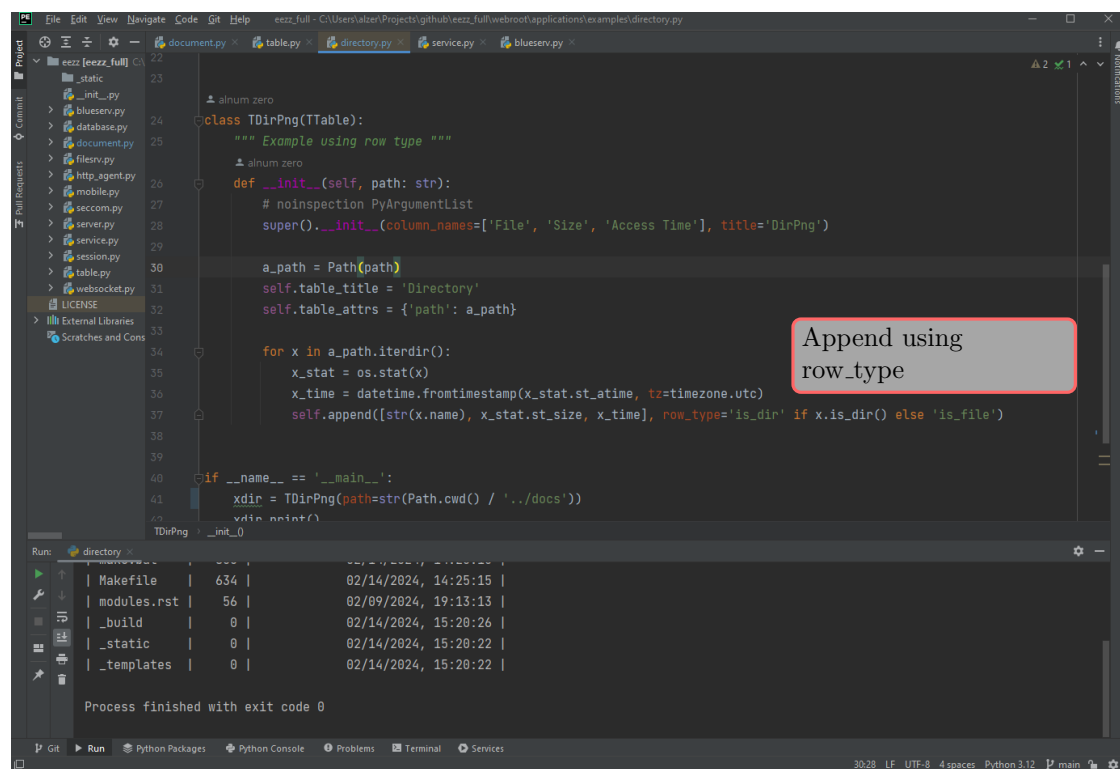


Figure 11: Example for Directory Listing

In the listing listing 21 you could see two templates with different match assignment. Each row type could define different actions for select and update.

```

43  <tbody>
44  <tr data-eezz="
45      template: row,
46      match: is_file">
47
48      <td></td>
49      <td class="clzz_{cell.type}"
50          data-eezz="template: cell">{cell.value}</td></tr>
51
52  <tr data-eezz="
53      template: row,
54      match: is_dir">
55
56      <td></td>
57      <td class="clzz_{cell.type}"
58          data-eezz="template: cell">{cell.value}</td></tr>
59  </tbody>

```

Listing 21: simple-tree1.html

For the ASCII output nothing changes. The template nodes could be interpreted like a switch statement. For each row the render machine will lookup the corresponding template. In our example one of the differences is the first cell, which loads individual icons, according to the row type as shown in the output in Figure 12

	File	Size	Access Time
📁	applications	4096	3.12.2024, 12:23:59
📁	database	4096	3.12.2024, 12:23:34
📁	logs	0	3.12.2024, 12:23:34
📁	public	4096	3.12.2024, 12:23:37
📁	resources	4096	3.12.2024, 12:23:34
📁	testdata	0	3.12.2024, 12:23:34
📄	__init__.py	0	27.12.2023, 15:35:43
📄	__pycache__	0	3.12.2024, 12:23:34

Figure 12: Listing with different Row Types

5.3 Tree Views

In the previous examples I have chosen a directory listing. With the class *TTableTree* it is also possible to show the entire directory tree and open/close the branches dynamically.

To do so, I defined the class *TDirTreeDetails(TTableTree)*, implemented the abstract method *TTableTree.open_dir* and adjusted the *HTML* page as shown in Listing 22. The method returns a *TTableRow* with a *TTableTree* child element.

```

64 <table id="Directory"
65   data-eezz='assign: examples.directory.TDirTreeDetails(title="Simple Tree", path="/home/user")'>
66   <thead>
67     <tr data-eezz="
68       template: row,
69       match: header">
70
71       <th></th><th
72         class="clzz_th"
73         data-eezz="
74           template: cell,
75           event: do_sort(column={cell.index}),
76           update: this.tbody">{cell.value}</th></tr>
77   </thead>
78   <tbody>
79     <tr data-eezz="
80       template: row,
81       match: is_file,
82       event: on_select(index={row.row_id}),
83       update: path_label.innerHTML = {row.row_id},
84             text_detail.innerHTML = read_file(path={row.row_id})"&>
85
86       <td></td>
87       <td class='clzz_{cell.type}'
88         data-eezz="template: cell">{cell.value}</td></tr>
89
90     <tr data-eezz="
91       template: row,
92       match: is_dir,
93       event: open_dir(path={row.row_id}),
94       update: this.subtree = this.tbody,
95             path_label.innerHTML = {row.row_id}">
96
97       <td></td>
98       <td class='clzz_{cell.type}'
99         data-eezz="template: cell">{cell.value}</td></tr>
100   </tbody>

```

Listing 22: simple-tree3.html

Important for the tree view is the update sequence in line 94 in Listing 22. This statement puts the sub-tree to the right place. The expression *this.subtree* is a predefined meta target and the value *this.body* defines the template to use.

The *JavaScript* implementation toggles open and close. The application could choose to keep the data.

In the example above, the directory node sends an update to the yellow text element to show the selected path and the file node triggers reading and displaying the files content, as shown in fig. 13.

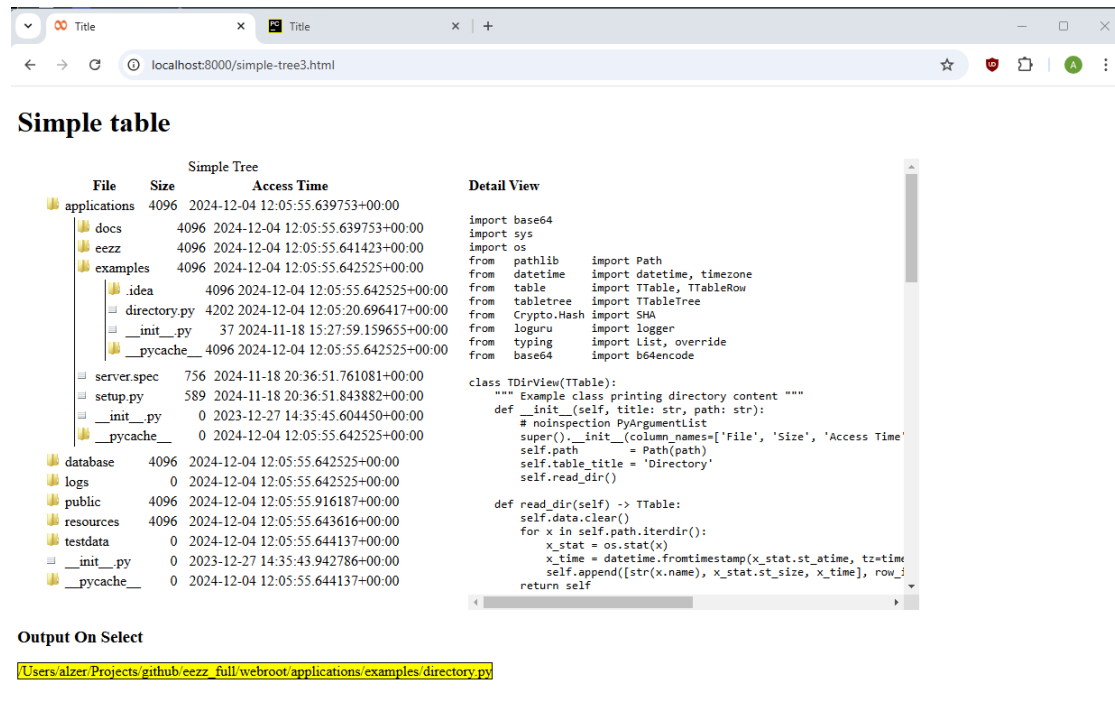


Figure 13: Example for Table and Details

Of cause, this would work with pictures as well.

5.4 Grid Views

The grid layout shows, that this pattern could be assigned to various types. We create a template tile, which is reproduced for each row of in the *TTable*. Essential for this to work is the class assignment to *class=clzz_grid*.

In the following the parent element has the style attribute *display:grid* and takes the role of the table structure in the previous examples and assigns a user method.

In the next level of the hierarchy you define an element, which defines the layout for each row in the table using the attribute *data-eezz="template:row"*. The third level of the hierarchy elements with attribute *data-eezz="template:cell(column-name)"* have access on cell values using the column name in brackets, which in our case is ("File", "Access Time", "Size"). In this example we place the cell values on relative positions within a tile. If the column name contains a space you need to specify quotes.


```

56 <div class="clzz_grid" id="Directory" data-eezz='assign: examples.directory.TDirView(title="Simple Tree", path="/Users/alzer/
    ↳ Projects/github/eezz_full/webroot")'>
57
58 <div class="grid-item" style = "position: relative;"
59   data-eezz = "
60     template: row,
61     match: body">
62
63     <span style = "position: relative; vertical-align: center; top: 20px"
64       data-eezz = "template: cell (File)">{cell.value}</span>
65
66     <span style = "position: absolute; left: 10px; top: 50px"
67       data-eezz = "template: cell ('Access Time')">{cell.value}</span>
68
69     <span style = "position: absolute; left: 30px; top: 100px"
70       data-eezz = "template: cell (Size)">{cell.value}</span>
71
72   </div>
</div>

```

Listing 23: Simple Grid Layout

The result is shown in the next picture fig. 14. Again it would be possible to define different tiles for different row types.

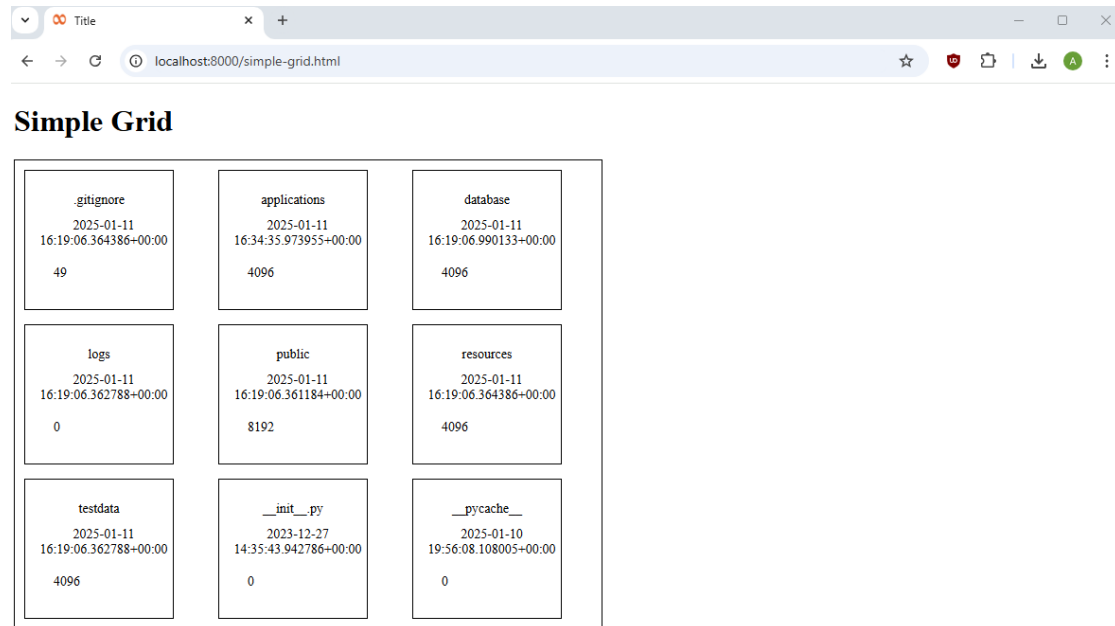


Figure 14: Example for Grid View

Now you could let your imagination run wild and give the grid tiles any outer appearance or add motion, connecting the transform properties of an elements with the back-end.

5.5 Form Input

Forms are implemented in this environment as a logical extension of the grid concept. I defined the class *TFormInput(TTable)* as shown in Listing 24 and set *TTable.visible_rows=1*. I want only one tile to become visible as input dialog.

One table row is needed to specify the data-types and depending on the user interface designer, using the given values as default or help. The result is shown in listing 25.

```

19 class TFormInput(TTable):
20     def __init__(self, title: str):
21         super().__init__(column_names=['Title', 'Description', 'Medium', 'Technique', 'Price'], title=title)
22         self.visible_items = 1
23         self.append(['', '', '', '', 0.0, ''])
24
25     def register(self, input_row: list) -> TTableRow:
26         # todo: implement register activities here
27         return super().append(input_row)
28
29
30 @dataclass

```

Listing 24: Python Script driving Form Input

In the *HTML* page, you specify the input tags and the submit button, as shown in listing 25. The elements of the form are collected using square brackets as selectors. The framework places all elements into the row in the correct order, allowing the back-end method to use the *TTable.append* method to store the incoming values.

```

30 <div class="clzz_grid" id="FormInput"
31     data-eezz='assign: examples.bookshelf.TFormInput(title="Simple Form")'>
32
33     <div style="display: grid; grid-template-columns: auto auto; gap:5px; width:300px"
34         data-eezz = "template: row" data-eezz-match="body">
35
36         <span>Title</span>
37         <input type = "text"
38             data-eezz = "template: cell (Title)"/>
39
40         <span>Description</span>
41         <textarea rows="4" cols="50"
42             data-eezz = "template: cell (Description)"/></textarea>
43
44         <span>Medium</span>
45         <input type = "text"
46             data-eezz = "template: cell (Medium)"/>
47
48         <span>Technique</span>
49         <input type = "text"
50             data-eezz = "template: cell (Technique)"/>
51
52         <span>Size</span>
53         <input type = "text"
54             data-eezz = "template: cell (Size)"/>
55
56         <span>Price</span>
57         <input type = "text"
58             data-eezz = "template: cell (Price)"/>
59
60         <input type = "button"
61             id = "commit_button"
62             value = "submit"
63             data-eezz = "event: register(input_row = [template.cell])"/>
64
65     </div>

```

Listing 25: Form Input HTML

As a result, you can define a form input with individual input tag elements. You can implement input checks in JavaScript within the browser or in the back-end. You can provide feedback about expected and submitted data, making this concept much more efficient and easier to implement than other approaches.

The screenshot shows a web browser window with the title 'Form Input' and the address bar displaying 'http://localhost:8000/simple-form-inp.html'. The page content includes a heading 'Simple Form Input' and a form with the following fields: 'Title' (text input), 'Description' (text area), 'Medium' (text input), 'Technique' (text input), 'Size' (text input), and 'Price' (text input). A 'submit' button is located at the bottom of the form.

Figure 15: Form Input

6 Compound Documents

A compound document is the skeleton of the EEZZ archive. Using this subsystem we could focus on the functions described above without being distracted by components not yet introduced. So the following examples demonstrate how to create such an compound document archive based on the features described above.

A compound document, like EEZZ, consists of a description, the Manifest, and a set of embedded files. The Manifest contains attributes for the document such as the author and creation date, as well as info for each file like name, type, hash, encryption, etc. Documents of the same structure are organized in a database like table, which is in this case called *bookshelf*, which is described in more detail in the following chapter.

6.1 Creating a Bookshelf

The class *eezz.TDocument* encapsulates the file handling, which includes download and management of compressed archives. The chunk-based, non-blocking download feature allows a feedback via a progress bar, which is particularly useful for large files.

In the first step I want to create a document, so I need an input dialog. The class *TSimpleShelf(TTable, TDocument)* maps the document attributes to table column as shown in in Listing 26.

```

30 @dataclass
31 class TSimpleShelf(TTable, TDocument):
32     shelf_name: str
33     attributes: List[str] = None
34     file_sources: List[str] = None
35     column_names: list = None
36     current_row: TTableRow = None
37     id: str = ''
38
39     def __post_init__(self):
40         """ Initialize the hierarchy of inheritance """
41         # Adjust attributes
42         self.attributes = ['Title', 'Header', 'Description', 'Medium', 'Technique', 'Price', 'Size', 'Status']
43         self.file_sources = ['main', 'detail']
44         self.id = self.shelf_name
45         TDocument.__post_init__(self)
46
47         # Set the column names and create the table
48         self.column_names = self.attributes
49         TTable.__post_init__(self)
50
51         self.visible_items = 1
52         self.append(table_row=['', '', '', '', 0.0, '', ''], row_type='input')

```

Listing 26: Bookshelf Example

The process of generating a document is separated in three phases. Phase one saves the attributes into the *TTable*, phase two is the download process of files and the last phase generates an archive with all the input.

The cascade is triggered with the statements in the commit button in Listing 27. The *data-eezz:event* is executed. For the update target there is a function call instead of an element attribute. An update target is always executed on the browser, so this is a call to a *JavaScript* function, with the argument compiled to `"read_files({files:[ main, detail ] })"`.

This is an EEZZ function, included with *websocket.js* and it organizes the download. The method uses the statement shown in Listing 28 to evaluate the recipient of the data. It shows also how an update of a progress bar could be implemented. Important is the "sync" option in this case, so that the progress bar outcome is not random.

```

126 <input type="button" value="submit"
127       data-eezz=" "
128       event: prepare_document(values=[template.cell]),
129       update: read_files(files=[main, detail])"/>

```

Listing 27: Trigger Download

The referenced *HTML* tags listed in the call to *read_files* contain the download instruction, which is in this case is the method *TDocument.download_file*. The download has two arguments: a descriptor and a binary stream and the download object has two attributes *file* and *byte-stream*. The method is executed on the back-end and returns a percentage value as *UTF-8* encoded byte-stream, to update the progress bar.

```

112 <div style="display: grid; grid-template-row: auto auto; row-gap: 4px;">
113   <input
114     type = "file"
115     data-eezz="
116       template: cell (detail); environment: process = sync,
117       update:
118         progress_detail.style.width = download_file(file=this.file, stream=this.bytestream),
119         progress_detail.innerHTML = progress_detail.style.width"/>
120
121   <div style="background-color:red">
122     <div id="progress_detail"
123       style="width: 5%; background-color:green">5%</div></div>
124 </div>

```

Listing 28: File Download Snippet

The dialog would look as follows:

The screenshot shows a web browser window with the address bar displaying 'localhost:8000/compound-doc.html'. The page title is 'Generate Compound Document'. The form contains the following fields:

- Title:** Text input with value 'FloraMay1'.
- Header:** Text input with value 'Flora:May I'.
- Description:** Text area with value 'dickicht
blaues gezweig
kreucht und fleucht
schwere reife im abendlicht
ruhe'.
- Medium:** Text input with value 'Leinwand'.
- Technique:** Text input with value 'Acryl, Kreiden'.
- Size:** Text input with value '80x120'.
- Price:** Text input with value '1200'.
- Status:** Text input with value ''.
- Main Picture:** File selection button 'Datei auswählen' with value 'flora may_1.jpg' and a progress bar showing 5% completion.
- Detail Pictures:** File selection button 'Datei auswählen' with value 'flora may_1_4.jpg' and a progress bar showing 5% completion.

A 'submit' button is located at the bottom left of the form.

Figure 16: Form Input for Bookshelf Paintings

The manifest and all files are zipped into a resulting compound document. For any part of an *HTML* page, you can now reference such a document to create content.

```

DATABASE
  Paintings
    FloraMay1.tar
      flora may I.jpg
      flora may I_2.jpg
      flora may I_3.jpg
      flora may I_4.jpg
      Manifest
      {
        "document": {
          "Title": "FloraMay1",
          "Header": "Flora:May I",
          "Description": "dickicht\n\nblaues gezweig\n\nkr\u00e4ucht und fleucht\n\nschwere reife im abendlicht\n\n
            ↳ nruhe",
          "Medium": "Leinwand",
          "Technique": "Acryl, Kreiden",
          "Price": "1200",
          "Size": "80x120",
          "Status": null,
          "main": [
            "flora may I.jpg"
          ],
          "detail": [
            "flora may I_2.jpg",
            "flora may I_3.jpg",
            "flora may I_4.jpg"
          ]
        },
        "files": [
          {
            "source": "main",
            "name": "flora may I.jpg",
            "size": 196284,
            "type": ""
          },
          {
            "source": "detail",
            "name": "flora may I_2.jpg",
            "size": 316425,
            "type": ""
          },
          {
            "source": "detail",
            "name": "flora may I_3.jpg",
            "size": 422578,
            "type": ""
          },
          {
            "source": "detail",
            "name": "flora may I_4.jpg",
            "size": 522286,
            "type": ""
          }
        ]
      }
    }
  }

```

Listing 29: Resulting Data Structure

This approach makes it much easier to make changes, add or remove content.

6.2 Display a Bookshelf

To display the compound document, we need to load the data from local storage, which is achieved with the statement *data-eezz:oninit*. The method specified in this statement is executed before rendering.

```

42 <div class="clzz_grid" id="bookshelf"
43   data-eezz='
44     assign: examples.bookshelf.TSimpleShelf(shelf_name="Paintings"),
45     oninit: load_documents()'>

```

Listing 30: Assign with Initialization

The Manifest for compound documents, stored in the archives, are designed in a way, that restoring a document from data stream could be done in just a few lines.

```

70 def load_documents(self) -> TTableRow:
71     """ Load the bookshelf from scratch """
72     self.clear()
73     self.visible_items = 20
74
75     x_path = self.path / self.shelf_name
76     for x in x_path.glob('*.tar'):
77         self.manifest.loads(self.read_file(x.stem, 'Manifest'))
78         x_row_values = self.manifest.document.values()
79         self.selected_row = self.append(x_row_values, row_id=x_row_values[0])
80     return self.selected_row

```

Listing 31: Load Bookshelf

Images could be loaded either using browser source path or using a back-end method defined by the *data-eezz:onload* statement.

```

76 <div class="pictures_panel">
77     <img style="max-width:500px; height:auto"
78         data-eezz = "
79             template: cell (main),
80             onload:
81                 this.src = read_file(
82                     document_title = {cell.id},
83                     file_name = {cell.value})"/>

```

Listing 32: Loading Images from Archive

Looking at the result keep in mind, that this is a template for all documents in a bookshelf. You could restrict the number of tiles via *TTable.visible_items*, you could sort, filter and navigate. You could manage the content of the bookshelf by adding, removing or simply filter the entries in the content directory.



Display Compound Document

Flora:May I

dickicht
blaues gezweig
kräucht und fleucht
schwere reife im abendlicht
ruhe

Medium	Leinwand
Technique	Acryl, Kreiden
Size	80x120







Figure 17: Bookshelf with first Tile

I made use of a the format directives for the text above. The `data-eezz:format` statement gives the framework a hint to replace double newlines either with "`<br>`" *line-break* or "`<p>`" *paragraph*.

Compare this with the project [WordPress ZedlitzArt](#). I achieved the same results with my new approach, but with significant less overhead and less time for adding new pictures.

7 Summary and Perspective

In this project, I built a user interface that can create and show a compound document, which is the backbone of an EEZZ archive.

The full EEZZ project includes many other parts, such as handling Bluetooth connections, encryption, key management, and purchase processes. These are all handled on the server side and would need a separate paper to explain. The user interface I created is designed to work well with all these other parts as well.

One highlight was how well input forms worked in this setup. They didn't need complicated code to save data to the database and could even supports default values and data sets. I didn't expect this to work so smoothly when I started.

Another important part was that users can view documents directly from the archive without needing to first extract parts of the content. This made it easy to add decryption and verification while the data is being streamed, keeping the entire structure less vulnerable.

Using templates in HTML was my key invention. This is what makes it interesting for other interactive applications. It made designing complex pages easier without needing special HTML generating JavaScript code, which is always a nightmare to maintain and often fragile in runtime and support. On the other side templates are very robust and I could even show the same data in different ways (like a table or tiles) without changing the server site code. This kept the HTML pages neat and easy to read.

It took for example only one HTML element definition to integrate Python's `loggeru` package into the user interface, which gives detailed information about what the server is doing and how long each step takes. This was very helpful while developing the system. So from the very beginning I could track the performance.

I was able to connect the various elements on one screen, collecting input and distributing output. I also successfully re-implement the project in SAP-ABAP to show that this concept would even hold in a commercial environment.

As a next step I will investigate into an extension for moving elements using SVG and transformation syntax. This would open some new areas for simulation and gaming. Another aspect

would be the integration of two or more applications using special templates, something which is very often requested as user extension for a given application.

You can find the code and documentation here: github.com/albert-zero/eezz_next.